SECURITY AUDIT REPORT

CodeAlpha Internship — Task 3: Secure Coding Review

Application	CodeAlpha Secure Coding Review Lab
Technology	Node.js, Express, SQLite
Reviewed artifact	src/server.js (70 lines)
Audit approach	Static source-code review
Assessment status	Preliminary code review — runtime tests not performed
Auditor	[Your name]
Date	[Insert submission date]

Important: This report analyzes the deliberately vulnerable training application supplied in the project ZIP. It is not a certification, penetration-test report, or claim that runtime tests have passed. Findings below are based on the source code and line references.

1. Executive Summary

A static review of the supplied Express API identified multiple security weaknesses affecting credential storage, SQL query construction, access control, authentication protections, input validation, and error handling. The application is explicitly designed as an educational vulnerable lab and binds to the loopback interface (127.0.0.1). It must remain local and must not be deployed or exposed to a network.

The highest-priority issues are SQL injection in the login and user-lookup queries, and plaintext password storage. The profile endpoint also returns records without authenticating or authorizing the requester. The login endpoint checks credentials but does not establish a session or issue a token, so it does not provide a complete authenticated session flow.

2. Scope and Application Overview

In-scope files reviewed: **src/server.js**, **package.json**, **README.md**, and **docs/AUDIT_CHECKLIST.md** from the uploaded ZIP. The backend provides health, registration, login, user lookup, and profile endpoints. SQLite database initialization and the demo account are defined in the server source.

Out of scope: deployment configuration, operating-system security, production infrastructure, third-party service integrations, and any system other than this local training application. No external target was tested.

3. Methodology and Limitations

The review consisted of reading the supplied source and tracing how request values reach database queries and API responses. Line numbers refer to the supplied **src/server.js**. The project was not executed during preparation of this report; therefore, no runtime exploit, Postman result, automated scanner result, or successful remediation retest is claimed. Before final submission, run the application locally, reproduce findings safely, capture screenshots, and update the retest fields.

4. Findings Summary

ID	Finding	Severity	Evidence
F-01	SQL injection in login query	High	41, 66
F-02	SQL injection in user lookup	High	50–52
F-03	Passwords stored in plaintext	High	15, 16, 29
F-04	Profile endpoint lacks authentication/authorization	Medium	56–62
F-05	No login rate limiting	Medium	37–45
F-06	No input validation at registration	Medium	26–30
F-07	Internal error details exposed	Low–Medium	31–34, 44, 53, 62
F-08	Demo credentials embedded in source	Medium	16

5. Detailed Findings

F-01 — SQL Injection in Login Query [HIGH]

Location: src/server.js, lines 40–44; query construction at line 41; helper at line 66

Observation: The login query concatenates username and password values into SQL using dbQuote(). dbQuote() only surrounds the value with quote characters; it does not escape quotes or use bound parameters. Request-controlled values can therefore alter the SQL statement.

Potential impact: An attacker able to submit login requests may manipulate the query logic and potentially bypass authentication or disclose account data, depending on the final SQL expression and SQLite behavior.

Recommended remediation: Replace string concatenation with a parameterized query using placeholders. Store password hashes, retrieve the account by username using a bound parameter, then verify the submitted password with a password-hashing library such as Argon2id or bcrypt. Remove dbQuote().

Verification status: Code-level issue confirmed by inspection. Runtime reproduction and secure retest: NOT PERFORMED.

F-02 — SQL Injection in User Lookup [HIGH]

Location: src/server.js, lines 48–53; especially line 51

Observation: The username query parameter is concatenated directly into a SQL statement inside the /users endpoint. The value is not passed as a bound parameter.

Potential impact: A crafted username query value may change the WHERE clause and expose records beyond the intended username match.

Recommended remediation: Use a parameterized statement, for example SELECT ... WHERE username = ?, with username supplied as a bound parameter. Also enforce authorization if this endpoint is intended to expose user information.

Verification status: Code-level issue confirmed by inspection. Runtime reproduction and secure retest: NOT PERFORMED.

F-03 — Plaintext Password Storage [HIGH]

Location: src/server.js, lines 15–16 and 28–30

Observation: The users table stores the password in a column named password, the seeded demo account uses a readable password, and registration inserts the submitted password directly.

Potential impact: If the SQLite database or a backup is accessed, every stored user password is immediately readable. Users who reuse passwords could suffer compromise on other services.

Recommended remediation: Never store plaintext passwords. Hash passwords using Argon2id or bcrypt with an appropriate work factor and unique salt. Verify hashes during login. Remove plaintext demo credentials from source and reset any training data after testing.

Verification status: Code-level issue confirmed by inspection. Hashing implementation and retest: NOT PERFORMED.

F-04 — Missing Authentication and Authorization on Profile Endpoint [MEDIUM]

Location: src/server.js, lines 56–62

Observation: GET /profile/:id returns a user profile based only on the ID in the URL. No authentication middleware or ownership/admin authorization check is applied.

Potential impact: Any local caller who can reach the endpoint can request another user ID and read the returned id, username, and role. This is an insecure direct object reference / broken access control pattern.

Recommended remediation: Require a valid authenticated identity, then authorize that identity to access the requested record. Permit cross-user access only for explicitly authorized administrative roles. Return only fields required by the caller.

Verification status: Code-level issue confirmed by inspection. Authorization tests: NOT PERFORMED.

F-05 — No Login Rate Limiting [MEDIUM]

Location: src/server.js, lines 37–45

Observation: The login route has no per-account or per-source throttling, progressive delay, or other brute-force protection.

Potential impact: An attacker may submit repeated password guesses. The impact depends on deployment and other controls; the current lab is loopback-only.

Recommended remediation: Apply a rate limiter to authentication routes, use sensible thresholds and temporary backoff, monitor repeated failures, and consider account-safe abuse controls. Avoid permanent lockouts that enable denial of service.

Verification status: Absence of rate-limiting middleware is visible in the reviewed source. Load/abuse testing: NOT PERFORMED.

F-06 — Missing Registration Input Validation [MEDIUM]

Location: src/server.js, lines 26–30

Observation: Registration reads username and password from the request body and passes them to the database without checking that they are strings or enforcing length, format, or password policy.

Potential impact: Malformed or unexpectedly large values may cause errors, poor data quality, or abuse of the registration endpoint. The JSON parser has a 20 KB body limit, but that is not a substitute for field validation.

Recommended remediation: Validate request schema and types; enforce reasonable length limits and username rules; apply a documented password policy; reject unexpected fields; and return consistent client errors.

Verification status: Missing field-level validation is visible in the reviewed source. Boundary testing: NOT PERFORMED.

F-07 — Internal Database Errors Exposed [LOW–MEDIUM]

Location: src/server.js, lines 31–34, 44, 53 and 62

Observation: Several catch blocks return err.message directly to the client. Database errors can reveal implementation details such as table or constraint information.

Potential impact: Detailed errors can help an attacker understand the database structure and refine attacks. The exact information disclosed depends on the error.

Recommended remediation: Return generic error messages to clients, log diagnostic details securely on the server, and ensure logs do not contain passwords or sensitive request bodies.

Verification status: Error response behavior is visible in the reviewed source. Runtime error-path tests: NOT PERFORMED.

F-08 — Demo Credentials Embedded in Source [MEDIUM]

Location: src/server.js, line 16

Observation: The source seeds a demo account with a known username and password in a database initialization statement.

Potential impact: If the account remains enabled outside a disposable local lab, anyone who knows the source can attempt to authenticate with the known credentials.

Recommended remediation: Keep demo accounts disabled by default in production. Load any required test credentials from local-only configuration, rotate them, and ensure test fixtures are excluded from production builds.

Verification status: Credential literal confirmed by inspection. Environment/deployment review: NOT PERFORMED.

6. Remediation Plan

Priority	Action	Acceptance criteria
1	Replace every SQL string concatenation with parameterized queries.	All request-derived SQL values are bound parameters; SQL injection tests do not alter query intent.
2	Implement password hashing and remove plaintext demo credentials.	Database contains only password hashes; valid login verifies a hash.
3	Add authentication and object-level authorization.	Unauthenticated profile access is rejected; users cannot access other users' records unless authorized.
4	Add login rate limiting.	Repeated login attempts are throttled and behavior is documented.
5	Validate all request bodies and query/path parameters.	Invalid types, missing values and out-of-range lengths are rejected.
6	Sanitize client-facing errors.	Clients receive generic messages; diagnostic details remain in protected server logs.
7	Move demo/test fixtures out of production paths.	No known demo credential or test account is active in production.

7. Retesting and Evidence

This document is a preliminary source-code audit. No application execution, Postman testing, scanner run, exploit reproduction, or remediation verification was performed while preparing it. Do not mark findings as fixed until a corrected implementation has been tested. Add dated screenshots and actual request/response evidence to the final version.

Suggested safe local retest checklist

• Confirm the server listens only on 127.0.0.1.
• Test ordinary registration and login with fictional accounts.
• Verify that stored credentials are hashes, not original passwords.
• Submit SQL-special-character inputs and confirm parameterized queries preserve intended lookup behavior.
• Confirm unauthenticated profile requests are denied and users cannot access other users' records.
• Make repeated failed login attempts and confirm throttling.
• Submit invalid field types and oversized values and confirm validation errors.
• Trigger a controlled server-side error and confirm stack/database details are not returned.

8. Conclusion

The reviewed source contains multiple clear security weaknesses and is suitable as a controlled training target for CodeAlpha Task 3. The most important fixes are parameterized SQL, secure password hashing, and proper authentication/authorization. This report should be updated after the application is run, findings are safely reproduced, fixes are implemented in a separate secure version, and retests are completed.

Appendix — Source Notes

The supplied README identifies the application as an intentionally vulnerable local Express + SQLite lab and warns against deployment. The source binds to 127.0.0.1 (line 68), which is a useful containment measure; it does not make the application safe for public exposure. The project's own audit checklist lists the principal review areas.

Prepared for educational use — CodeAlpha Cybersecurity Internship, Task 3.